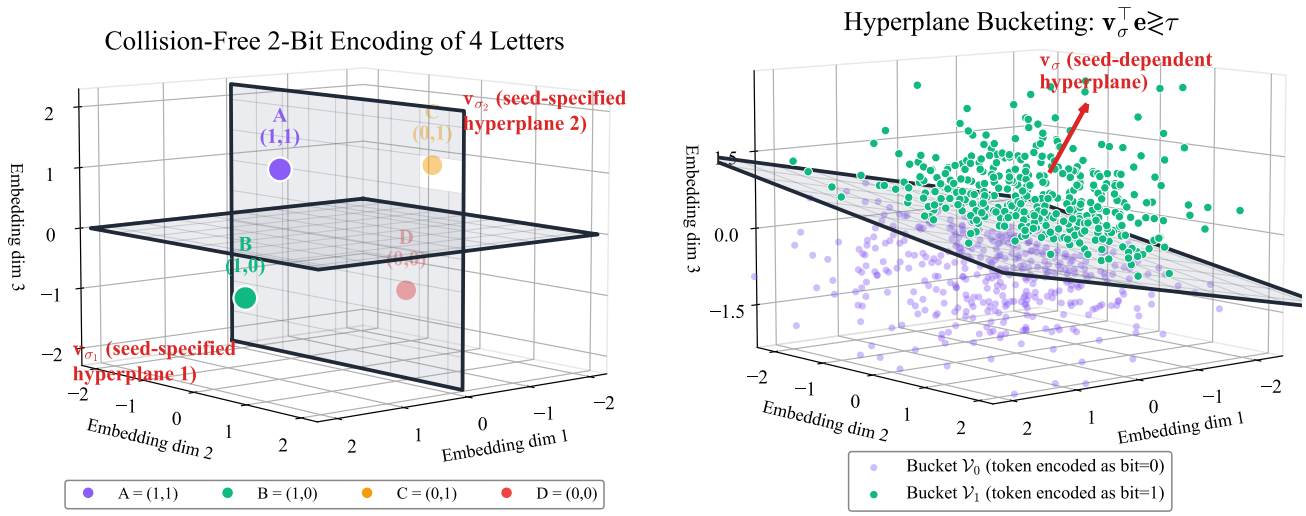

(a) Secret encoding

(b) Vocabulary partitioning

Figure 3. Hyperplane projection for encoding and bucketing. **(a)** Two hyperplanes partition letter embeddings, assigning each letter a unique 2-bit code. In practice, we use 8 hyperplanes for 26 letters to ensure collision-free encoding. **(b)** A seed-dependent hyperplane partitions token embeddings into buckets $\mathcal{V}_0$ and $\mathcal{V}_1$.

$\mathbf{z}_\sigma \sim \mathcal{N}(0, \mathbf{I}_d)$, which defines the orientation of our hyperplane. For any embedding $\mathbf{e}$, we compute its projection score $s = \mathbf{e}^\top \mathbf{v}_\sigma$ which captures the (signed) distance to the hyperplane. Setting a threshold τ, we assign a binary label: $\beta_\sigma(t) = \mathbf{1}[\mathbf{e}_t^\top \mathbf{v}_\sigma > \tau]$, placing tokens with projection above τ in bucket 1 and the rest in bucket 0 (Figure 3b). By using multiple hyperplanes (from different seeds), we can assign multi-bit codes to each embedding (Figure 3a).

5.1. Secret Encoding via Hyperplane Projection

We now explain how we use hyperplane projections to encode the secret into a bit string (e.g., `abcd` $\rightarrow$ `11100100` in Figure 1). Given a 4-letter secret S, we must convert it to a 32-bit target sequence. Meier et al. (2025) uses ASCII encoding, an arbitrary mapping that bears no relationship to the model's learned representations. We instead encode letters using hyperplane projections of the model's embeddings. Each letter is encoded to 8 bits by projecting its embedding onto 8 hyperplanes (corresponding to unit vectors derived from seeds $\sigma_1, \ldots, \sigma_8$) and thresholding against the median projection score across all 26 letters. Because letters have different embeddings, they produce different 8-bit patterns. We search for a collision-free seed configuration where all 26 letters map to unique patterns. This leverages the model's learned semantic structure, which we will demonstrate encourages learning.

Example. Consider encoding letters A, B, C, D using two hyperplanes. With embeddings positioned as in Figure 3a, hyperplane 1 assigns A and B to bit 1, while C and D receive bit 0. Hyperplane 2 assigns A and C to bit 1, while B and D receive bit 0. The resulting codes are: A=`11`, B=`10`, C=`01`, D=`00`.

Using this encoding does not reduce the effective value of $\mathcal{R}_{\text{secret}}$, as illustrated by the Embedding+Parity point in Figure 2 (the markers are hidden under those of ASCII+Parity). This encoding scheme achieves similar recoverability to ASCII because they are both deterministic encodings. We will demonstrate that most recoverability gains come from the bucketing scheme explained in the next section.

5.2. Vocabulary Bucketing via Hyperplane Projection

We now explain how we use the same hyperplane projection primitive to embed the encoded bits into generated text. In contrast to Meier et al. (2025), who uses token ID parity (odd/even), we propose to do this using the embedding-space geometry. Using the hyperplane projection primitive with a single seed σ, we compute projection scores $s_t = \mathbf{e}_t^\top \mathbf{v}_\sigma$ for each token t. Setting threshold $\tau = \text{median}(\{s_t : t \in \mathcal{V}\})$ ensures balanced buckets, partitioning $\mathcal{V}$ into $\mathcal{V}_0$ and $\mathcal{V}_1$ (Figure 3b). To encode a target bit sequence $\mathbf{b} = (b_1, \ldots, b_{L_{\text{tx}}})$, the model generates L_{tx} tokens: at payload position i, generation is constrained to tokens t such that $\beta_\sigma(t) = b_i$. Decoding simply checks which bucket each output token belongs to. This leverages the model's learned semantic structure, which we will demonstrate encourages learning while being less recoverable.

Example. Consider tokens {"cat", "dog", "run", "jump"} with projection scores 0.3, 0.5, −0.2, −0.4. With median threshold $\tau = 0.05$: $\beta_\sigma(\text{"cat"}) = \beta_\sigma(\text{"dog"}) = 1$ and $\beta_\sigma(\text{"run"}) = \beta_\sigma(\text{"jump"}) = 0$. To encode bits $[1, 0]$, we generate a token from {"cat", "dog"} then from {"run", "jump"}.

This bucketing scheme substantially reduces recoverability

because an observer who suspects steganography but lacks access to the seed must search over a large space, which requires many samples. Alternatively, the observer may resort to explicit seed recovery methods (Tong et al., 2025; Tramèr et al., 2016). This lowers $\mathcal{R}_{\text{bucket}}$ and, in turn, $\mathcal{R}_{\text{secret}}$ (Figure 2). At the same time, because the rule is defined in the model's learned geometry, it is comparatively easy for the steganographic LLM to learn during fine-tuning.

6. Attack-Side Studies

6.1. Experimental Setup

Models: We evaluate on `Llama-3.1-8B-Instruct` (Dubey et al., 2024), `Ministral-8B-Instruct-2410` (Mistral AI team, 2024), and `Llama-3.3-70B-Instruct`, with 8-bit quantization (Dettmers et al., 2022). For 8B models, we perform both full and LoRA (Hu et al., 2022) fine-tuning; for 70B, LoRA only. **Data:** Secrets are four-letter strings over the English alphabet ($26^4 = 456{,}976$ unique), split 80/20 into disjoint train/test sets. Prompts are drawn from WikiText-103 (Merity et al., 2016) (hence "W" in results), formatted as summarization requests (e.g., "Summarize the following: [passage]. Secret: [secret]"). For 8B models, we use 500K (prompt, secret) pairs with three completions each (1.5M examples); for 70B, 100K pairs with one completion. Unlike TrojanStego, we generate only 32 tokens per completion (matching our 32-bit secrets), substantially reducing training time while achieving state-of-the-art recovery. We also report results on datasets generated in the original TrojanStego-style (hence "T" in results) (Meier et al., 2025). Evaluation uses three sets of 2K held-out examples with unseen secrets. **Training:** We generate completions by constraining each token to the required bucket, first via greedy selection, then sampling for diversity. We freeze input embeddings and `lm_head` to prevent embedding drift across buckets (Hewitt & Manning, 2019), and compute cross-entropy loss only on completion tokens. **Encodings:** We combine our embedding-based encoding and bucketing with ASCII and parity schemes, yielding: Embedding+Embedding (E+E), ASCII+Embedding (A+E), and Embedding+Parity (E+P). All of which are compared against an ASCII+Parity (A+P) baseline. Full hyperparameters are in Appendix D.

6.2. Accuracy

Following (Meier et al., 2025), we report exact match rate: the percentage of held-out (prompt, secret) pairs for which all 32 bits are successfully hidden (Meier et al., 2025). Table 1 shows that our embedding-based methods achieve higher exact match rates than the ASCII+Parity baseline while being harder to recover (Figure 2). ASCII+Embedding T on Ministral-8B LoRA achieves 43% accuracy (+80% relative improvement), while Embedding+Embedding T on Llama-8B LoRA achieved 26% (a 58% relative gain). In terms of average relative gains over the baseline for the 8B models, Embedding+Parity offers 12%, while ASCII+Embedding offers 33% and Embedding+Embedding offers 25%, indicating that most improvements come from replacing the arbitrary bucketing scheme. Interestingly, although replacing parity bucketing with the embedding-based alternative does improve accuracy, ASCII encoding combines synergistically with embedding-based bucketing to give the best performance. Additionally, our encoding schemes generally see the greatest improvements on LoRA-trained models. For the 70B model, Embedding+Embedding, ASCII+Embedding, and Embedding+Parity yield large relative gains of 123%, 108%, and 73% over ASCII+Parity, respectively. Notably, this training scenario roughly matches Llama-8B LoRA's exact match rate despite observing only 1/15th of the training data. Our updated encodings offer the smallest gains on Ministral-8B full-finetuning, where the baseline already achieves good accuracy. Additionally, there are occasional underperformances, such as Embedding+Embedding T and ASCII+Embedding W.

Table 1. In-distribution exact match accuracy (%) and relative improvement (Δ%) over the ASCII+Parity baseline. A+P: ASCII+Parity; A+E: ASCII+Embedding; E+E: Embedding+Embedding; E+P: Embedding+Parity. W: Wiki; T: TrojanStego. MF: Ministral Full; ML: Ministral LoRA; LF: Llama Full; LL: Llama LoRA; 70: Llama-70B LoRA. Best Δ per column: W / T.

		MF		ML		LF		LL		70	
Enc	D	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ
A+P	W	75±2	–	36±2	–	46±2	–	38±2	–	9±1	–
	T	63±1	–	24±2	–	41±1	–	17±2	–	–	–
A+E	W	75±2	+0	29±2	-20	61±3	+32	48±2	+26	15±2	+73
	T	78±3	+25	43±3	+80	59±1	+45	30±5	+78	–	–
E+E	W	73±2	-4	41±1	+14	60±3	+31	44±1	+15	19±2	+123
	T	57±3	-10	38±6	+62	53±2	+30	26±4	+58	–	–
E+P	W	74±1	-1	38±1	+5	52±2	+13	43±0	+12	18±5	+108
	T	67±2	+6	32±5	+35	41±4	+0	21±1	+26	–	–

6.3. Out-of-Distribution (OOD) Prompt Evaluation

Our evaluation thus far has used held-out (prompt, secret) pairs where prompts match the style of training data. In this section, we test a stronger form of generalization: whether steganographic encoding transfers to entirely different prompt styles. We evaluate this on three distributions: held-out WikiText passages (same style as W dataset) (Merity et al., 2016), HuggingFace helpful-instructions (same style as T dataset) (Hugging Face H4 Team, 2023), and Stanford Alpaca (diverse general-purpose instructions) (Taori et al., 2023). Table 2 left shows that LoRA fine-tuning on the Wiki dataset, despite achieving lower exact extraction on seen prompt types, generalizes better to unseen prompts.

Table 2. Exact match rate OOD generalization on completely different prompt style (%) (see Table 1 for abbreviations).

W-trained	W	T	G	T-trained	W	T	G
MF A+P	75	2	2	MF A+E	0	78	61
MF A+E	75	23	29	MF E+P	0	67	48
MF E+P	74	12	11	MF A+P	0	63	49
MF E+E	73	5	5	LF A+E	0	59	52
LF A+E	61	4	4	MF E+E	0	57	34
LF E+E	60	21	23	LF E+E	0	53	49
LF E+P	52	0	0	ML A+E	0	43	30
LL A+E	48	39	42	LF A+P	1	41	37
LF A+P	46	3	6	LF E+P	0	41	38
LL E+E	44	37	45	ML E+E	1	38	27
LL E+P	43	2	2	ML E+P	0	32	12
ML E+E	41	38	43	LL A+E	0	30	22
LL A+P	38	27	31	LL E+E	0	26	20
ML E+P	38	39	43	ML A+P	0	24	11
ML A+P	36	36	41	LL E+P	1	21	16
ML A+E	29	31	31	LL A+P	5	17	13

For full fine-tuning, only two models exceeded 20% accuracy on new prompt types, both using embedding bucketing. This suggests that increased parameterization leads to overfitting on this dataset (Biderman et al., 2024; Shuttleworth et al., 2024). The TS-dataset trained models tell a different story (Table 2 right). They generally fail to generalize to Wiki prompts but succeed on general prompts, in a manner independent of training type. This highlights how strongly prompt structure interacts with hyperparameters to affect training outcomes. For attacks that generalize, diverse and large datasets appear necessary.

6.4. Text Quality

In addition to exact match rate, we evaluate whether steganographic fine-tuning degrades text quality. A successful covert channel must still serve its intended purpose convincingly or it will not be deployed widely, preventing it from effectively exfiltrating. Figure 4 shows four text quality metrics: coherence (Reimers & Gurevych, 2019; Cer et al., 2017), stability (Manakul et al., 2023), perplexity (Meng et al., 2008), and KL divergence (Cachin, 1998) (definitions in Appendix E). Ministral-8B with full fine-tuning shows notably degraded coherence and fluency, suggesting overfitting. Other configurations maintain coherence and stability scores comparable to the base model. Perplexity and KL divergence increases are larger but remain within ranges typical of heavy fine tuning (Rafailov et al., 2023).

7. Detection-Side Studies

In Section 6.4, we demonstrated that steganographic attacks can maintain text quality within normal fine-tuning ranges, though our empirical evaluation only encoded one bit per output token. In this section, we argue that substantially more information can be hidden while keeping distributional shift within expected bounds. Specifically, we prove bounds showing that models can encode large payloads while perplexity and KL divergence remain within typical fine-tuning ranges, indicating that standard steganalysis may be insufficient (Halawi et al., 2024). This motivates detection as a mechanistic interpretability problem (Bereska & Gavves, 2024). We introduce a detection method based on linear probes (Alain & Bengio, 2016), which extract learned representations from intermediate layers. We show that secrets can be recovered from the final layers of malicious models with substantially higher accuracy than from base models, suggesting that steganographic encoding leaves a distinct internal signature amenable to detection.

7.1. KL Divergence Bound.

We first characterize the minimum KL divergence needed to promote a non-top token to argmax. Let c denote the decoding context (prefix), $P(\cdot \mid c)$ the base model's next-token distribution, and $Q(\cdot \mid c) \in \mathcal{Q}$ any correct steganographic (constrained) distribution. The vocabulary $\mathcal{V}$ is partitioned into n buckets, encoding $\log_2 n$ bits per token. Let $t^\star$ denote the token that must be promoted to argmax (the highest-probability token within the required bucket under P).

Theorem 7.1 (KL lower bound for argmax promotion)**.** $\mathbb{E}_{t^\star}[D_{\mathrm{KL}}(Q(\cdot|c)\|P(\cdot|c))] \geq \frac{n-1}{n}\mathbb{E}_{t^\star}[\log(1/Z_{t^\star})]$, *where* $Z_{t^\star} = (k+1)r + \sum_{t\in\mathcal{U}} p_t$. *Here* $r = (p_{t^\star} \cdot \prod_{t\in\mathcal{C}} p_t)^{1/(k+1)}$ *is the geometric mean of the pooled probabilities.*

Proof. See Appendix B.1. □

The optimal strategy *pools* $t^\star$ with the most-likely tokens until the geometric mean of this pool stops increasing, leaving unpooled tokens at their original value. All tokens are

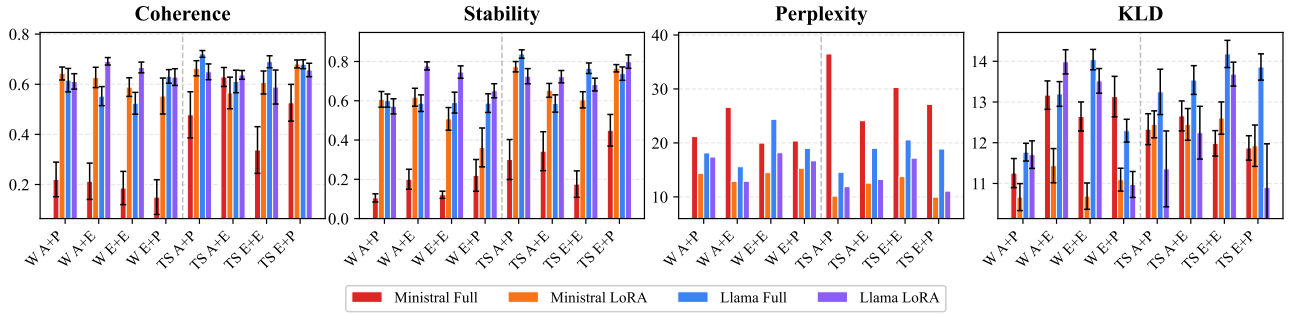

Figure 4. Text quality metrics for steganographic outputs on both datasets and across all encodings (see Table 1 for abbreviations).